# Platform Engineer

## Assessment Demo Guide

Customer file retrieval | Google Cloud + Terraform + Python FastAPI

### Purpose

Use this guide to present the architecture, walk through the Terraform and Python implementation, run the live request demonstration, and discuss what remains to be validated.

### The project in one sentence

A FastAPI service on Cloud Run authorizes an exercise customer request and retrieves that customer's file from a private Cloud Storage bucket using a dedicated runtime identity.

### What is deployed

Cloud Run with the Python image, a private versioned bucket, Artifact Registry, a runtime service account, Secret Manager token injection, and customer ID restriction. The deployed endpoint was tested through the Cloud Run proxy.

### What is configured but not deployed

Terraform includes optional global load balancer and Cloud Armor resources. They remain disabled. The HTTPS option needs a domain; the temporary HTTP mode has no TLS and should not carry real credentials or customer data.

### Assessment targets

Stage 1: 6,000+ requests/second, 99.9% success, and 99.9% under 300 ms. Stage 3: 99.99% success and 99.99% under 300 ms. These are targets, not results proven by this sample.

A good demo is honest about both the working path and the remaining work.

# Start with the supplied diagram

Use the interview's drawing as the base. Explain the translation: Cluster Ingress becomes the Cloud Run request entry point; the File-service Pod becomes the Cloud Run FastAPI service; Google Compute Disk is replaced with private Cloud Storage for file objects. The current implementation has no database pod.

## Walk the target request path

1. Customer/API client sends a request. The target design places HTTPS Load Balancing and Cloud Armor at the edge.
2. The load balancer routes to Cloud Run through a regional serverless NEG. This optional infrastructure is not deployed yet.
3. FastAPI validates the customer and file identifiers, checks the API key, and restricts this exercise token to demo-customer.
4. The Cloud Run runtime service account reads the object from the private bucket.
5. Structured request logs capture method, route, status, and duration.

## Important distinction

The architecture sketch includes OIDC/JWT, metadata ownership lookup, GitHub Actions Workload Identity Federation, and a secondary region. Those are proposed extensions. The current service uses a static exercise API key scoped to one configured customer; it does not validate JWTs, use a metadata database, or fail over to another region.

## Miro legend

Solid arrows: request and file data flow. Dashed arrows: permissions, provisioning, or planned components. Mark Cloud Run, GCS, IAM, and Secret Manager as deployed; mark the load balancer/Armor as configured but not deployed; mark JWT, database, CI federation, and regional failover as future.

# Terraform walkthrough: foundation

## Files: terraform/versions.tf and terraform.tfvars.example

versions.tf requires Terraform 1.5+ and pins the Google provider family to ~> 6.0; the provider uses the chosen project and region. terraform.tfvars.example supplies the assessment project ID and europe-west2 region as a starting configuration. Keep real credentials and state out of Git.

## File: terraform/main.tf

google_project_service.required enables the APIs Terraform needs: Artifact Registry, Cloud Resource Manager, Compute, IAM, Cloud Run, Secret Manager, and Cloud Storage. disable_on_destroy=false leaves APIs enabled when the stack is destroyed.

google_storage_bucket.files creates the customer file bucket. Uniform bucket-level access simplifies IAM, public access prevention blocks public exposure, versioning keeps object versions, and force_destroy=false protects against deleting a non-empty bucket.

google_artifact_registry_repository.service creates the Docker repository used to store the application image.

google_service_account.runtime creates a dedicated identity for the running service. google_storage_bucket_iam_member.read_files grants it roles/storage.objectViewer on this bucket only, rather than broad project access.

## Secret Manager resources

The secret value is created separately with gcloud, not Terraform. data.google_secret_manager_secret.api_token looks up the secret by name. google_secret_manager_secret_iam_member.api_token_accessor grants the runtime identity secretAccessor. The secret value is injected into Cloud Run as API_TOKEN and is not placed in Terraform configuration/state as a literal.

## google_cloud_run_v2_service.api

Deploys the selected container image in europe-west2. The template attaches the runtime service account, sets a 30-second request timeout, concurrency 80, min/max instances 1/100 by default, 1 CPU, 512 MiB memory, port 80 for Nginx or 8080 for FastAPI, GCS_BUCKET, API_TOKEN_CUSTOMER_ID, and the optional API_TOKEN secret reference.

The Cloud Run ingress is INGRESS_TRAFFIC_ALL while the optional LB flags are off. When an LB mode is enabled, Terraform changes ingress to INGRESS_TRAFFIC_INTERNAL_LOAD_BALANCER.

google_cloud_run_v2_service_iam_member.public_invoker grants allUsers run.invoker only for the public Nginx flag or when the LB is enabled. With the LB enabled, restricted ingress still prevents direct external access to the run.app endpoint; the application must continue enforcing its own authorization.

# Terraform walkthrough: optional load balancer

## File: terraform/load_balancer.tf

When either optional flag is true, Terraform can create a static global IP, a regional serverless NEG pointing to Cloud Run, a Cloud Armor security policy, a backend service with request logging, and the URL map/proxy/forwarding rule for the selected frontend.

## Cloud Armor rules

The policy enforces low-sensitivity OWASP SQL injection and XSS WAF rules, then allows other traffic by default. It has no rate-limit rule. Logs and false positives should be reviewed before production use.

## HTTPS mode

enable_external_https_lb=true requires lb_domain to be a hostname you control and a DNS A record pointing to the reserved IP. Terraform creates a Google-managed certificate and redirects HTTP to HTTPS. Certificate activation can take time after DNS resolves.

## HTTP-only mode

enable_external_http_lb=true can demonstrate the edge without a domain, but it sends the API key and files over unencrypted HTTP. It is for temporary, disposable test data only. In this project it was planned but not applied.

## Files: terraform/variables.tf and outputs.tf

Variables set project, region, image, port, bucket name, scale bounds, concurrency, authentication, secret name/version, and the authorized exercise customer. The external LB flags default false. Outputs expose the Cloud Run URL, bucket, runtime service account, Artifact Registry path, and optional LB IP/URL.

This Terraform is a starting implementation, not evidence of 6,000 requests per second. The actual LB plan showed 9 to add, 1 in-place update, 0 destroy; it was not applied.

# Python walkthrough: request lifecycle

## Files: platform-file-service/app/main.py and config.py

config.py reads API_TOKEN, GCS_BUCKET, LOG_LEVEL, and API_TOKEN_CUSTOMER_ID from the environment. The configured customer defaults to demo-customer.

main.py defines GET /v1/customers/{customer_id}/files/{file_id}. It validates the path IDs with allow-list regular expressions, checks X-API-Key using hmac.compare_digest, rejects other customer IDs with 403, then reads customers/{customer_id}/{file_id} from Cloud Storage.

## Responses

401: missing or incorrect API key. 400: invalid identifiers. 403: key is not authorized for that customer. 404: object/fixture not found. 413: file exceeds the 10 MiB example cap. 502: generic storage/backend failure. Successful responses set private, no-store, attachment filename, and nosniff headers.

## Storage path

With GCS_BUCKET set, the service calls blob.reload() for metadata and then download_as_bytes(). This gives a size check before downloading, but adds a metadata request and buffers the file in memory. For large files or strict tail latency, measure realistic payloads and consider streaming or short-lived authorized download URLs.

## Health and observability

/health/live reports process liveness. /health/ready returns 503 when no API token is configured. HTTP middleware emits a JSON request-completed log with method, route, status, and duration; it avoids recording token values or file content. This is structured logging, not a complete custom metrics/tracing/SLO-alert setup.

## Container: Dockerfile

Uses Python 3.12 slim, installs pinned runtime dependencies, runs Uvicorn on port 8080, and switches to non-root UID 10001. .dockerignore keeps local development files out of the build context.

# Suggested demo sequence

## 1. Open Miro

Show the supplied diagram as the base and describe the target request path. State explicitly that the HTTPS LB/Cloud Armor is present in Terraform but not deployed; JWT, DB, CI federation, and secondary region are future work.

## 2. Walk Terraform

Point out the private bucket, limited runtime service account, Secret Manager reference, Cloud Run service, scaling starting values, and optional LB file. Do not claim the optional LB was applied.

## 3. Walk Python

Show main.py: key check, input validation, configured customer boundary, GCS object path, file size limit, response headers, generic backend error, and structured request logging.

## 4. Run the live request

```
Terminal 1: gcloud run services proxy customer-file-service --region=europe-west2 --
project=assessment-512510 --port=8081
Terminal 2: curl -i -H 'X-API-Key: <DEMO_TOKEN>' http://localhost:8081/v1/customers/demo-customer/files/
welcome.txt
```

Expected: 200 OK and the test object's content. This was previously observed as 'Hello from GCS'.

## 5. Show tenant boundary

Repeat the curl with /v1/customers/another-customer/files/welcome.txt. Expected: 403 and 'Not authorized for this customer'. This was previously observed. The exercise key is scoped to one configured customer, not a production identity system.

Security note

Use the Cloud Run proxy for the live demo, not the HTTP-only public LB mode. Do not put the API token in slides, screenshots, or a public repository.

# Tests, SLOs, and interview answers

## Automated tests: tests/test_api.py and tests/conftest.py

The suite covers health endpoints, API-key required/incorrect responses, local fixture success, missing fixture, identifier validation, GCS object path and download, oversize rejection before download, backend failure hiding sensitive detail, and missing GCS object. Run from platform-file-service with the development requirements installed: pytest -q. Only claim it passed if you ran it successfully in your environment.

## What the live test proves

The Cloud Run proxy request proved the deployed service returned the uploaded GCS test file (HTTP 200). The other-customer request proved the configured customer check returned HTTP 403. These are functional integration checks, not load or availability tests.

## SLO and scale discussion

Cloud Run concurrency 80 and max instances 100 are configuration values, not proof of 6,000 RPS. Load test with representative object sizes, burst/steady traffic, regions, storage latency, quotas, and enough load generators. Measure success rate and the fraction under 300 ms directly; track p95/p99 latency, errors, saturation, and resource use. Stage 3 targets are stricter than Stage 1.

## Security improvements

Replace the shared exercise key with signed customer JWT/OIDC validation (issuer, audience, expiry, signature, scope/customer claim); keep Cloud Run ingress behind the LB; use least-privilege identities; rotate secrets; add rate limits and review Cloud Armor logs. The present app scopes one shared token to one customer ID only.

## Resilience and separation

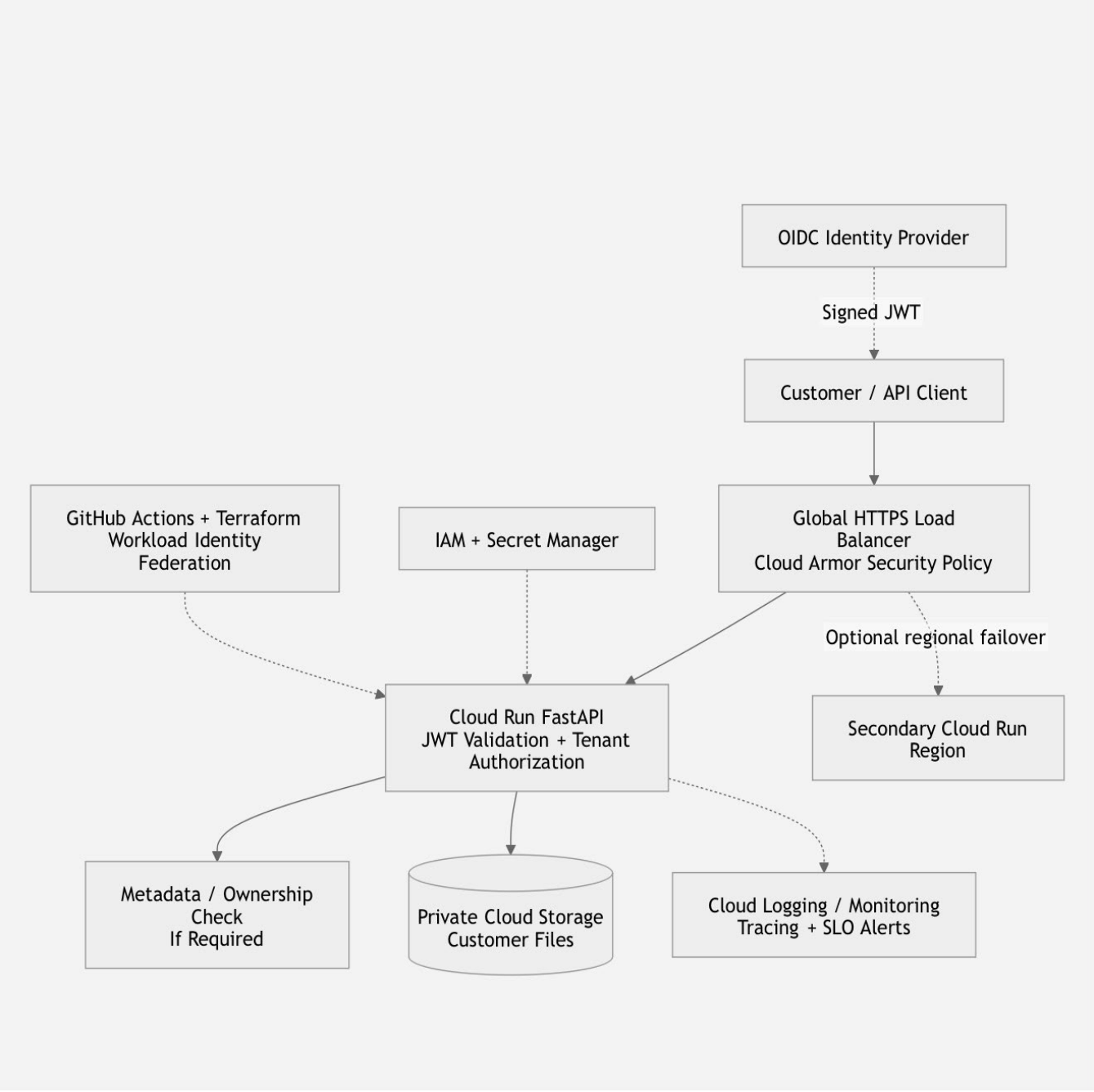
Cloud Run scales stateless service instances; GCS stores durable file objects; metadata belongs in a database only if the product needs ownership/search metadata. Add multi-region service and storage/failover only with tested recovery objectives. Keep file retrieval synchronous; move audits, scanning, conversion, or upload processing to Pub/Sub with idempotency, retries, and dead-letter handling.

## Closing line

This is a small, testable foundation. I would next replace the exercise token with customer JWT auth, deploy the LB/Armor with a domain, add SLO dashboards and traces, then validate capacity and failover under representative load.

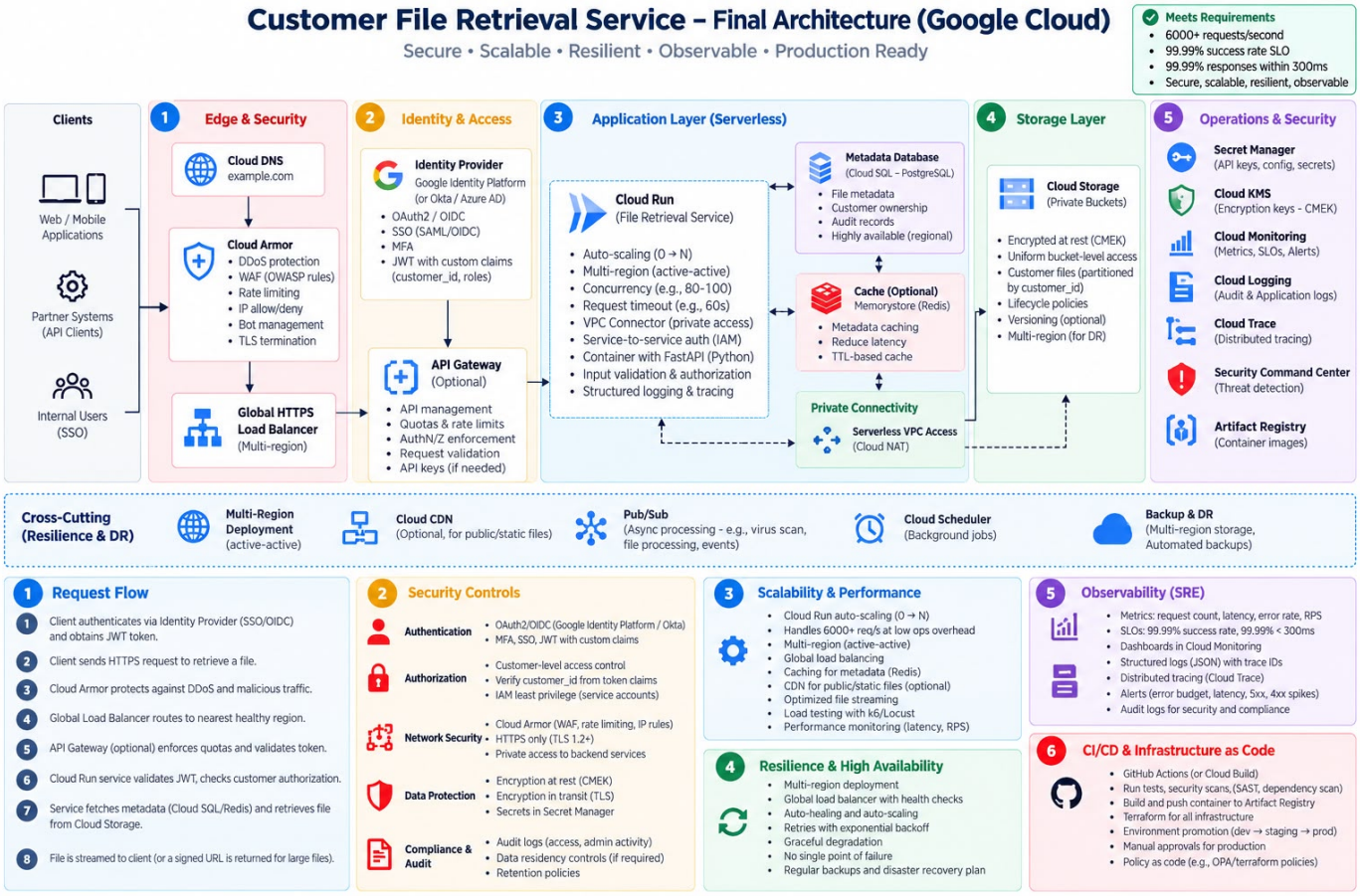
# Target architecture reference

Use this as a discussion aid. The diagram includes proposed components that are not part of the current



# Expanded target architecture

This is a proposed production direction. Green checkmarks and SLO figures in the source image are design goals, not measured results from this assessment deployment.




In the demo, distinguish what is deployed (Cloud Run, private GCS, IAM, Secret Manager, Artifact Registry) from optional Terraform (load balancer/Cloud Armor) and future target components.

# How to explain the expanded diagram

## Start with the purpose

This is a target design for retrieving customer files securely at high traffic. It separates the request path into edge protection, identity, application, data, and operations. The current assessment implementation is a smaller slice of that target.

## 1. Clients and edge security

Web/mobile apps, partner API clients, and internal users send HTTPS requests. Cloud DNS maps a hostname to the service. Cloud Armor at the global HTTPS load balancer can apply WAF rules, IP controls, and configured rate limits before requests reach the application. The Terraform contains an optional load balancer and basic SQLi/XSS WAF policy, but these resources are not deployed. It does not currently configure rate limiting, bot management, or a custom domain.

## 2. Identity and access

An identity provider authenticates the user or client and issues a signed token. The API or API Gateway must validate signature, issuer, audience, expiry, and relevant claims. Cloud Armor is an edge filter; it does not perform the application's customer authorization. The current Python service uses an exercise API key and restricts it to the configured demo-customer ID. JWT/OIDC, SSO, and MFA are target improvements, not implemented features.

## 3. Application and metadata

Cloud Run runs stateless FastAPI containers and can add instances as demand rises. The app validates IDs, checks the configured customer boundary, then reads the matching object from GCS. A metadata database is optional if the product needs ownership records, search, or audit metadata. Redis can cache frequently used metadata, but should not become the source of truth for file authorization. Neither Cloud SQL nor Memorystore is deployed in this project.

## 4. File storage and private connectivity

Cloud Storage is the durable file-object store. The bucket currently has uniform bucket-level access, public access prevention, object versioning, and a narrowly scoped object-viewer grant for the runtime service account. CMEK, multi-region storage, lifecycle rules, and VPC private access are target options, not current settings. The existing bucket is in europe-west2.

## 5. Operations and supply chain

Secret Manager supplies the API token to Cloud Run; Artifact Registry stores the container image. Cloud Run logs and the app's structured JSON request logs provide basic operational visibility. A production version should add SLO dashboards and alerts, distributed tracing, audit review, stronger secret rotation, image/dependency scanning, and a controlled CI/CD pipeline. KMS, Security Command Center, custom tracing/alerts, and GitHub Actions Workload Identity Federation are not implemented here.

## 6. Cross-cutting resilience

Pub/Sub is suitable for work that can happen after the file response, such as audit enrichment, malware scanning, or conversions. Scheduler can run periodic housekeeping. CDN is appropriate only for public or carefully authorized static content. Multi-region service failover and backup/recovery need explicit RTO/RPO decisions and exercises. None of these optional components is currently deployed.

# Numbered request flow to narrate

1. The client signs in with the identity provider and obtains a token. In this project, use the demo API key instead; the service does not validate JWTs.

2. The client sends an HTTPS request for a customer and file. A production hostname resolves through Cloud DNS to the global HTTPS load balancer. The optional load balancer is currently disabled and not deployed.

3. Cloud Armor evaluates configured edge rules. The Terraform example includes SQLi/XSS WAF rules but does not currently add rate limits or bot controls.

4. The load balancer routes to a healthy regional backend. For Cloud Run this uses a serverless NEG. Restricted Cloud Run ingress is configured only when the optional LB mode is enabled; today ingress is all networks, while IAM invocation remains required for the Python deployment.

5. FastAPI validates the path, authenticates the exercise key, and checks that the requested customer matches the configured customer ID. The demonstrated other-customer request returned 403.

6. The Cloud Run runtime service account accesses only the configured file bucket. The service constructs customers/{customer_id}/{file_id}, checks metadata, and downloads up to the 10 MiB example limit.

7. The service returns the file with private/no-store and attachment/nosniff headers, while emitting a structured request log. A production path might stream large files or return a short-lived signed URL after authorization.

8. Optional async tasks can be published to Pub/Sub after the response; retries, idempotency, and dead-letter handling belong in the worker design.

## Correct SLO and scale statement

The source diagram labels 6,000 RPS and 99.99% success/latency as met. Do not repeat that as a result. The assessment's Stage 1 target is 99.9% success and 99.9% under 300 ms; Stage 3 asks for 99.99% success and 99.99% under 300 ms. No load test in this project proves either target or 6,000 RPS. Cloud Run settings (min 1, max 100, concurrency 80, 30-second timeout) are starting configuration only. Validate with realistic file sizes, traffic shape, load generators, storage latency, quotas, and regional failure tests.

## What is implemented versus shown

Implemented: single-region Cloud Run FastAPI, private versioned GCS bucket, runtime service account with bucket read access, Secret Manager token injection, Artifact Registry, API-key check plus one-customer restriction, health endpoints, structured request logs, and unit/integration test code. Previously demonstrated live: expected file returned 200 and another customer returned 403. Optional but not deployed: global load balancer and Cloud Armor policy. Future: JWT/OIDC, API Gateway, DB/cache, VPC connector, CDN, Pub/Sub/Scheduler, CMEK/KMS, multi-region active-active/failover, tracing/SLO alerts/SCC, and CI/CD federation.

A strong closing: 'This diagram is my target architecture. I implemented and manually verified the core Cloud Run-to-private-GCS retrieval path. I would add the edge layer and production identity controls, then validate SLOs and failover before claiming the capacity targets.'